

기술 설계 리뷰 · 알고리즘의 근거

파이프라인을 외우게 하지 않는다 — 왜 이렇게 설계됐는지 이해시킨다

F2-A는 동적 디스패치가 지운 **액션 → 핸들러** 매핑을 증거 기반 계산으로 복원한다.

이 발표는 각 결정이 **왜 필요하고, 왜 단순 대안보다 나은지**를 다룬다.

문제 > **왜 이 구조인가** > 왜 이 논리가 옳은가

직접 호출은 콜그래프에 있고, 간접 호출은 없다

질문 — "RemoteStartTransaction"은 어느 C 함수가 처리하나?

```
remote_handler(f);                // 직접 호출 — 대상 이름이 코드에 있음 → 콜그래프 0

Entry e = g_table[ f->action ];   // 테이블에서 함수 포인터를 꺼내
e.fn(f->payload);                 // 간접 호출 — 대상은 런타임 값 → 콜그래프 x
```

문제 우리가 필요한 건 "RemoteStartTransaction" → remote_handler() 라는 간선이다.

단순 대안 콜그래프를 따라간다 → 간접 호출 지점엔 대상 간선이 아예 없어 끊긴다.

그래서 호출이 아니라 “그 포인터가 어디서 채워졌나”(등록)를 봐야 한다.

등록 지점 → 핸들러 후보 → 최종 해석

먼저 — 이 발표에서 쓰는 다섯 용어를 코드 위에서 정의한다.

```
g_table[ACTION_REMOTE_START] = (Entry){ .fn = remote_handler }; ① 등록 지점
...
Entry e = g_table[f->action]; e.fn(...); ② 간접 호출(디스패치)
```

직접 호출	callee 이름이 코드에 있음 — 콜그래프가 안다
간접 호출	저장된 포인터를 호출 — 대상은 런타임 값
등록 지점	액션 ID ↔ 콜백이 데이터로 기록되는 곳 (①)
핸들러 후보	F2-A가 제안하는 (콜백, 근거) 쌍 — 아직 확정 아님
최종 해석	후보 중 선택된 하나 또는 보류

추출기가 곧바로 “이게 핸들러”라 하면 왜 안 되나

질문 — 관찰과 결정을 왜 분리하나?

직접 결정 (단순 대안)

추출기가 콜백을 찾음 → 즉시 “핸들러” 선언

- × 두 추출기가 불일치
- × 중복 관찰을 여러 번 계산
- × 신뢰도 정책이 추출기마다 박힘
- × 최종 답을 일관되게 감사 불가

증거 기반 (F2-A)

관찰 → 타입 증거 방출 → 공유 계산이 결합 → 판정 계층 하나가 확정/보류

- ✓ 충돌 처리 · 중복 제거
- ✓ 교차 확증 · 추적성
- ✓ 새 추출기로 확장



증거 추상은 모듈화가 아니라 일관된 추론의 필요조건이다 — 정책·중복·충돌·감사를 한곳에 모은다.

같은 액션의 여러 표기를 하나의 정체성으로

질문 — 표기가 제각각인데 같은 액션인지 어떻게 아나?

문제 — 표기만 보면 무관해 보인다

```
"RemoteStartTransaction" // 와이어 문자열
ACTION_REMOTE_START      // 매크로
3                         // 숫자 ID
remote_start              // 별칭
    ↓ KB
RemoteStartTransaction    // 하나의 정규 정체성
```

- EXACT** KB가 그 심볼을 명시적으로 지정 — 가장 강함
- RESOLVED** 상수 평가가 한 단계 필요했음 → 조금 약함
- NORMALIZED** 정규화가 구분을 뭉갤 수 있음 → 더 약함
- SUBSTRING** 무관한 이름도 부분 겹칠 수 있음 → 가장 약함

그래서 — 배수는 신뢰도의 순서

매치	x
EXACT_IDENTIFIER	1.00
RESOLVED_VALUE	0.95
NORMALIZED_NAME	0.85
HEURISTIC_SUBSTRING	0.70

같은 의미(action ↦ handler), 여러 문법

질문 — 인식기는 코드에서 무엇을 알아보나?

형태	코드	방출 증거
집합 초기화	{ ACTION, fn }	REGISTRATION_INIT
지정 초기화	{ .action = ACTION, .fn = fn }	REGISTRATION_INIT
인덱스 대입	handlers[ACTION] = fn	REGISTRATION_ASSIGN
상관 저장	t[k].action = ACTION; t[k].fn = fn	REGISTRATION_ASSIGN
등록 함수	register(ACTION, fn) → 콜리가 저장	REGISTRAR_CALL
디스패치	switch(action){ case ACTION: fn(); }	ENUM_CASE / STRING_DISPATCH

형태가 다르니 **단일 매치는 취약** — 형태마다 인식기를 두고, 결과는 하나의 증거 타입으로 통일한다

왜 이 두 필드를 “하나의 등록”으로 묶어도 되나

질문 — 어떤 상관이 안전하고, 어디서 멈춰야 하나?

안전 — 묶어도 됨

```
{ .action = ACTION_REMOTE_START,
  .fn      = remote_handler }
```

- 같은 aggregate 초기화에 속함
- .action.fn은 서로 다른 의미 필드
- 콜백이 내부 함수 심볼로 해석됨
- 객체 간 교차 짝짓기 없음 · 위치/부모 보존

위험 — 묶으면 안 됨

```
table[i].action = ACTION_REMOTE_START;
table[j].fn     = remote_handler;
// i == j 를 증명할 수 없음
```

같은 슬롯임을 정적으로 증명 못 하면 쌍을 만들지 않는다 → 잘못된 짝보다 보류가 옳다.

구조 상관은 “같은 슬롯”이 정적으로 확실할 때만 성립 — 규칙이 어디서 멈추는지가 정확성의 핵심이다.

모든 추출기의 출력 = 하나의 증거 레코드

질문 — 관찰을 정확히 어떤 형태로 남기나?

EVIDENCE (실제 객체)

```
Evidence {
  kind      : REGISTRATION_INIT
  callback  : remote_handler           // u.c:15
  action_id : { symbol: "ACTION_REMOTE_START",
                match : EXACT_IDENTIFIER }
  provenance : "site:reg:g_table"       // 그룹 키
  score      : 0.80                     // = 0.80 × 1.00
  records    : [ DISPATCH_HANDLER_TABLE u.c:12
                  HANDLER_REF           u.c:15 ]
}
```

각 필드가 왜 있나

kind → 사전확률(다음 장)

match → 매치 배수

provenance → 중복 제거·그룹화 키

score → prior × 배수

records → 파일·라인 = 감사 가능

여러 추출기가 동시에 방출 → 같은 바인딩에 대한 독립 관찰들이 계산 계층으로

곱셈은 “구조 AND 식별자” 게이트다

질문 — 왜 `prior × match` 인가? 더하면 안 되나?

$$\text{score} = \text{Prior}(\text{kind}) \times \text{MatchMult}(\text{strength})$$

구조(PRIOR)	식별자(MATCH)	곱	뜻
0.90	없음 0.00	0.00	액션 대응이 없음 → 증거 아님
0.30	1.00	0.30	정확 매치지만 구조 약함 → 제한적
0.80	1.00	0.80	구조·식별자 둘 다 강함

단순 대안 덧셈이면 `0.90 + 0.00 = 0.90` — 액션 대응이 전혀 없는데도 강해 보인다.

그래서 곱셈은 한 축이 0에 가까우면 전체를 0으로 끌어내린다 → 두 조건을 동시에 요구하는 논리적 AND.

prior = “이 방식이 action→callback을 얼마나 직접 표현하나”

질문 — 0.90 / 0.85 / 0.80 / 0.70 은 어디서 왔나?

방식	PRIOR	근거 — 얼마나 직접적인가
STRING_DISPATCH	0.90	비교와 호출이 한자리에 있음
ENUM_CASE	0.85	선택자와 구체 호출이 구조적으로 인접
REGISTRATION_INIT	0.80	액션·콜백이 명시적 짝, 그러나 호출은 딴 곳
REGISTRAR_CALL	0.70	콜리를 따라 종단 저장까지 가야 의미 성립
NAME_MATCH	0.70	어휘 유사성만 — 구조적 증명 아님

정직하게: 이 값들은 손튐닝된 보수적 정책 상수다. 통계적으로 보정된 확률이 아니며, 픽스처·평가로 검증된다.

한 등록의 레코드 3개는 증인 3명이 아니다

질문 — 같은 사실을 여러 번 보면 신뢰도가 커져야 하나?

그룹화 없이 - 과대계산

한 등록 `g_table` 가 남기는 레코드:

- 테이블 레코드 0.80
- 핸들러 참조 레코드 0.80
- 심볼 해석 레코드 0.80

$0.80 + 0.80 + 0.80 \rightarrow$ 부풀림

하나의 사실을 3가지로 본 것뿐

출처 그룹 - MAX

같은 등록 사이트

- 하나의 provenance 그룹
- **MAX = 0.80**

- 가장 강한 관찰이 공유 사실을 대표
- 약한 중복 뷰는 추적성은 더하되 독립 신뢰는 아님

중복을 독립 근거로 세면 어떤 결합식이든 신뢰도가 인위적으로 부풀다. 그래서 결합 전에 그룹 내부를 MAX로 접는다.

독립 그룹은 **noisy-OR**로 — 대안은 왜 나쁜가

질문 — 서로 다른 사이트의 확증은 어떻게 더하나?

결합식	문제
MAX	과대계산 방지 O · 독립 확증 무시 ×
SUM	확증 보상 O · 1 초과·과속 ×
AVERAGE	유계 O · 유효 약근거가 평균을 낮춤 ×
NOISY-OR	1 미만 유계 · 그룹마다 ↑ · 체감

$$\text{그룹 A} = 0.80, \quad \text{B} = 0.49 \quad 1 - (1-0.80)(1-0.49) = 1 - 0.20 \times 0.51 = \mathbf{0.898}$$

두 **약한 독립 근거**가 각자보다 커지되, 1엔 못 미친다

정직하게: 그룹을 **근사적으로 독립**이라 가정한 **공학적 결합 모델**이다 — 보정된 통계 확률이 아니다.

상한은 “쌓인 약함”과 “거짓 확실”을 막는다

질문 — 약한 근거가 여럿이면? 강한 근거가 여럿이면 확실한가?

WEAK-ONLY CAP = 0.85

이름 근거 0.70, 0.70, 0.70 noisy-OR → **0.973** (과신) 정확·해석 매
치 없음 → cap **0.85**

비슷한 약한 관찰 셋이 구조적 증명이 되진 않는다

GLOBAL CAP = 0.99

강한 정적 관찰이 여럿이어도 절대 확실은 아니다:

- 별칭을 놓칠 수 있음
 - KB가 불완전할 수 있음
 - 매크로·전처리가 구조를 가림
 - 런타임에 바뀔 수 있음
- 신뢰도는 **1.0** 미만 유지

두 상한 모두 모델이 실제보다 확신하는 것을 구조적으로 막는 보수적 장치다.

임계값과 마진은 서로 다른 질문에 답한다

질문 — 왜 최고 점수만 보면 안 되나?

수용 임계값 (FLOOR)
“1위가 애초에 충분히 강한가?” — 절대 강도

모호성 마진 (MARGIN)
“강하더라도 2위보다 분명히 나은가?” — 상대 우위

사례	1위	2위	이유	결과
A	0.42	—	임계값 미달	UNRESOLVED
B	0.66	0.56	둘 다 그럴듯, 차 $0.10 < 0.15$	AMBIGUOUS
C	0.72	0.56	강하고 차 $0.16 \geq 0.15$	RESOLVED

1위만 보면 사례 B에서 거짓 확신 — 0.66은 임계값은 넘지만 2위와 붙어 있어 단정할 수 없다.

확신에 찬 오답은 명시적 미지보다 해롭다

질문 — 왜 “모름”이 정확성 기능인가?

잘못된 확정의 하류 전파

잘못된 핸들러 해석



잘못된 오염 소스 → 잘못된 데이터 흐름



거짓 취약점 보고 · 또는 누락

명시적 보류

UNRESOLVED (사유 명시)



분석 공백이 드러남



근거 없는 하류 주장 없음

이것이 보수적 임계값·모호성 보존·구조화된 미해결 사유의 근본 이유다.

미해결엔 “왜 막혔는지”가 코드로 붙는다

질문 — 못 풀었을 때 무엇을, 왜 보고하나?

지원 — 슬롯이 정적으로 보임

```
t[k].action = ACTION; // id 저장
t[k].fn     = fn;     // 같은 슬롯 k
```

미지원 — 런타임 탐색 후 저장

```
for (i..) if (t[i].action == a) // 비교(슬어)
    { t[i].fn = fn; return; } // 심볼릭 i
```

미해결 사유 (구조화)

증거 없음	NO_EVIDENCE
임계값 미달	LOW_CONFIDENCE
저장 미도달	REGISTRAR_STORE_NOT_REACHED
탐색-후-저장	REGISTRAR_SEARCH_THEN_WRITE
외부 정의	EXTERNAL_DEFINITION
동적 액션 ID	DYNAMIC_ACTION_ID

id가 **저장이 아니라 비교**만 되고 슬롯이 심볼릭 i라 정적 쌍-저장이 닿지 못함 → 버그가 아니라 **명명된 범위 밖**

예제 — 소스에서 후보를 뽑아낸다

설정 — 같은 액션에 등록 후보와 디스패치 후보가 하나씩.

```
static Reg t[] = { { ACTION_DATA_TRANSFER, foo } };           // 등록 지점

switch (f->action){
    case ACTION_DATA_TRANSFER: bar();                          // 디스패치 지점
}
```

후보	사이트	관찰된 ACTION ID
foo	등록 (집합 초기화)	ACTION_DATA_TRANSFER
bar	디스패치 (switch case)	ACTION_DATA_TRANSFER

두 사이트 모두 같은 액션을 가리킨다 → 후보는 둘 (**foo, bar**). 아직 점수는 없다.

예제 — 증거 레코드와 점수

질문 — 각 필드는 어떻게 정해졌나?

```
foo:
  kind  = REGISTRATION_INIT    // 집합 초기화
  match = HEURISTIC_SUBSTRING  // 이름만 부분 겹침
  score = 0.80 × 0.70 = 0.56
```

```
bar:
  kind  = ENUM_CASE           // switch case
  match = NORMALIZED_NAME     // 정규화 후 일치
  score = 0.85 × 0.85 = 0.72
```

KIND 사이트 형태에서 결정 — 등록은 REGISTRATION_INIT, switch는 ENUM_CASE

MATCH 코드 식별자와 KB 정규 정체성의 대응도 (5장의 사다리)

SCORE prior × match — 디스패치가 더 강한 매치를 얻어 $0.72 > 0.56$

예제 — 결합 후 판정

질문 — 두 신뢰도로 무슨 결정이 나오나?

bar · 0.72

vs

foo · 0.56

수용: $0.72 \geq 0.50$ ✓ · 마진: $0.72 - 0.56 = 0.16 \geq 0.15$ ✓



RESOLVED → bar

두 검사(절대 강도·상대 우위)를 모두 통과 → 디스패치 핸들러 bar로 확정

조건을 바꾸면 거동이 어떻게 달라지나

질문 — 하나의 성공이 아니라 알고리즘의 거동을 본다.

바꾼 조건	점수	판정 논리	결과
switch 제거	foo 0.56 단독	$0.56 \geq 0.50$, 경쟁 없음	RESOLVED → foo
점수 근접	bar 0.62 · foo 0.56	마진 $0.06 < 0.15$	AMBIGUOUS
모두 약함	1위 0.42	$0.42 < 0.50$	UNRESOLVED
식별자 상충	×0.5 감점 (배수 cap 0.70)	점수 급락 → 임계값 미달	UNRESOLVED / 약화

같은 기계가 입력에 따라 **확정·모호·미해결**을 낸다 — 규칙이 결정하지, 예외 처리가 아니다.

미해결도 근거와 함께 실려 나가는 정식 결과

질문 — 후속 단계(F6)는 무엇을 받나?

```
"RemoteStartTransaction": {
  "status": "RESOLVED",
  "chosen": { "function": "remote_handler", "file": "u.c", "line": 15 },
  "candidates": [ { "function": "remote_handler", "confidence": 0.80,
                    "evidence_kinds": ["REGISTRATION_INIT"], "evidence": [ ... records ... ] } ]
},
"UpdateFirmware": {
  "status": "UNRESOLVED",
  "unresolved": { "reason": "NO_EVIDENCE", "attempted": ["registration_ast", "name_match"] }
}
```

handler_resolutions(권위·액션당 하나) + **handler_maps**(확정 전용·하위 호환) + **limitations**. 확정도 미해결도 근거로 추적된다.

무엇이 “구조적으로 참”이고, 무엇이 “정책”인가

정직성 — 보장되는 것과 조정 가능한 것을 구분한다.

구조 추출 규칙 (건전한 상관)

- 같은 초기화 안에서만 짝짓기
 - 콜백 = 내부 함수 심볼
 - 슬롯 동일성 정적 증명 필요
 - 위치·부모 구조 보존
- “추출 규칙은 구조적이다”

정책 파라미터 (검증되는 선택)

priors · match 배수 · Acceptance Floor · Ambiguity Margin · weak-only cap ·
global cap · conflict penalty
→ 손튜닝된 보수적 정책 상수

언어 “신뢰도는 순위 점수이지 보정된 확률이 아니다.”

언어 “임계값은 명시적·검증 가능한 정책 선택이다.”

언어 “평가는 그 정책이 기대 해석 거동을 내는지 확인한다.”

각 테스트 범주가 “무엇을” 검증하는가

질문 — 픽스처가 통과하면 무엇이 보장되나?

범주	검증 내용
역량 capability	지원 패턴이 의도한 증거·결과를 냄
충돌 conflict	경쟁 후보가 마진 부족 시 모호로 유지됨
거절 rejection	미지원 구조가 명명된 미해결 사유를 냄
회귀 regression	구현 변경이 신뢰도·상태·출처를 조용히 바꾸지 않음

구분: 구현 정확성 · 지원 문법 커버리지 · 실제 프로젝트 경험적 품질은 서로 다르다 — 단위 픽스처만으로 실세계 정확도가 증명되지 않는다.

전체 알고리즘 — 이제 “왜”까지 재구성할 수 있다

핵심 — 인식과 판정의 분리가 이 설계의 기여다.

```
# 액션 a
ev = dedupe( U extractor(a) )           # 인식 → 증거(관찰만)
for e in ev: e.score = Prior(e.kind) × MatchMult(e.match) # 곱 = 구조 AND 식별자
for cand in group_by_callback(ev):
    g    = { grp: max(score) }           # 중복 접기(과대계산 방지)
    conf = 1 - Π(1 - g[grp])             # 독립 확증(체감)
    if no strong match: conf = min(conf, 0.85) # 쌓인 약함 차단
top, second = rank_desc(cands)
return UNRESOLVED if top<0.50           # 충분히 강한가
    else AMBIGUOUS if top-second<0.15   # 분명히 나은가
    else RESOLVED(top)                  # 아니면 보류 — 오답보다 안전
```

각 줄의 주석이 “왜”다: 곱=AND 게이트, MAX=과대계산 방지, noisy-OR=독립 확증, cap=거짓 확실 차단, floor·margin=절대·상대, 보류=하류 오염 방지.

정책 상수 (검증되는 공학 파라미터)

상수	값	막는 실패
Acceptance Floor	0.50	약한 1위를 확정하는 것
Ambiguity Margin	0.15	붙어 있는 2위를 무시하는 것
weak-only cap	0.85	쌍인 약근거의 과신
global cap	0.99	정적 분석의 거짓 확실
conflict penalty	×0.5	식별자 상충(배수 cap 0.70)
registrar depth	2	등록 함수 무한 추적

모두 **정책 상수** — 수학적 증명이 아니라 보수적 선택이며 평가로 검증된다. (A2 출처 그룹화 · A3 지정 vs 집합 AST)